PROJECT DOCUMENTATION FOR THE ASSESSMENT ACTIVITY

Student Performance Analysis System

A Python project built with NumPy and Pandas

1. Project Title

Student Performance Analysis System – a Python project built with NumPy and Pandas.

2. Objective

Convert a simple list of marks into clear, ready-to-read results with no manual calculation.

Marks are often kept in a register or a spreadsheet, and totals, averages, grades and pass or fail status are then worked out by hand. This takes time, and a single small mistake can change a student's result. It is also hard to tell quickly who is leading the class or which subject needs more attention.

This project addresses that problem. It reads marks from a CSV file and calculates everything automatically for every student, then produces a ranked result list, a class summary, a subject-wise comparison and a list of high scorers. It also gave me a chance to practise Python, NumPy and Pandas on a realistic problem.

Prepared by	Mir Adnan Sadiq
Programme	B.Tech, 6th Semester
Branch	CSE
College	Government College of Engineering & Technology, Safapora, Ganderbal
Submitted to	Chandini Madam
Date of submission	20/10/2026

3. Technologies Used

Tool	Used for
Python	The language the entire project is written in.
Pandas	Loading the CSV, adding new columns, sorting, filtering and saving the results.
NumPy	Adding up and averaging each student's marks, and working out the subject averages.
CSV files	For the input data and the saved output.
Git and GitHub	For version control and for submitting the project.
VS Code	For writing and running the code.

4. Dataset Description

The data is stored in `students.csv` and contains records for 15 students, with one row per student.

Column	What it holds
Student_ID	A unique ID for each student (2501 to 2515)
Name	The student's full name
Department	CSE, IT or ECE
Python	Marks out of 100
Web Development	Marks out of 100
Cyber Security	Marks out of 100
Cloud Computing	Marks out of 100
Attendance	Attendance in percent

Four more columns are added by the program as it runs: `Total_Marks`, `Average_Marks`, `Grade` and `Result`.

5. Implementation

The code is divided into two files. `main.py` runs the whole program from start to finish, and `functions.py` contains the helper functions that it calls.

1. **Loading the file.** `main.py` first confirms that `students.csv` is in the same folder and shows a message if it is missing. If the file is found, it is loaded with `pd.read_csv()` and the student count, column names and first five rows are displayed. It also checks that every required column is present and that all marks and attendance values are valid numbers from 0 to 100.
2. **Totals and averages.** `add_totals()` selects the four subject columns, converts them into a NumPy array, and calculates the sum and average of each row. The average is rounded to two decimal places.
3. **Grade and result.** `get_grade()` turns the average into a grade, and `get_result()` is applied to each row to decide Pass or Fail.

Average	Grade
90 and above	A+
80 and above	A
70 and above	B
60 and above	C
50 and above	D
Below 50	F

> A student is marked Pass only if **every subject mark is at least 40** and **attendance is at least 40%**.

4. **Ranking.** Students are sorted by average marks, from highest to lowest, and listed with their total, average, grade, attendance and result.
5. **Class summary.** The program works out the class average, finds the top and lowest scorers with `idxmax()` and `idxmin()`, and counts the passes and fails to get the pass percentage.
6. **Subject analysis.** `subject_summary()` returns the average, highest and lowest mark for each subject, and the subject with the best class average is displayed.
7. **High scorers.** Students whose average is 80 or above are filtered out and listed.
8. **Saving.** The complete results are written to `student_results.csv` and the subject table to `subject_analysis.csv`.

6. Key Features

Feature	What it does
File check	Displays a clear message instead of crashing when students.csv is missing
Data checks	Reports missing columns and invalid marks before any calculation starts
add_totals()	Calculates total and average marks using NumPy
get_grade()	Assigns each student a grade from A+ to F
get_result()	Decides Pass or Fail based on marks and attendance
subject_summary()	Returns the average, highest and lowest mark of every subject
Ranking	Lists all students from the highest to the lowest average
Class summary	Shows the class average, topper, lowest scorer and pass percentage
High scorers	Lists students with an average of 80 or above
Saved output	Saves the results to two CSV files

7. Final Outcome

Running `main.py` now produces a complete performance report within seconds, and it also saves the processed data so it can be opened in Excel.

For the 15 students in the dataset, the results were as follows:

69.03 Class average	**73.33%** Pass percentage (11 passed, 4 failed)	**83.75** Highest average Dev Malhotra (A)	**53.25** Lowest average
2 Students with an average of 80 or above	**75.60** Best subject: Cyber Security (class average)		

The project achieves what it set out to do: it replaces manual calculation with a program that is fast, repeatable and easy to read.

8. Challenges & Learning

- A missing column in the CSV made the program stop with a `KeyError`. I added `check_columns()` in `functions.py` so `main.py` lists exactly which columns are missing and exits cleanly.
- Marks typed as text or outside the 0 to 100 range gave wrong totals. `clean_marks()` now converts the values with `pd.to_numeric()`, and the program stops and shows the offending rows if any entry is invalid.
- Grade boundaries needed care, because a student with exactly 80 must get an A and not a B. I used >= checks in strict top-to-bottom order so each boundary falls in the correct grade.
- An empty CSV or an empty high scorers list could break the output. I added checks that print a clear message instead of an error or a blank table.

What I learned: working with data in Pandas, calculating with NumPy arrays, sorting and filtering a DataFrame, writing reusable functions, organising a project across multiple files and uploading it to GitHub.

9. GitHub Repository

Repository link: https://github.com/Adee4-u/student-performance-analysis